

Verilog Language and Application

Week 5 Lab Document

Lab Manual

Revision 1.0

© 1990–2024 Cadence Design Systems, Inc. All rights reserved.

Printed in the United States of America.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. (Cadence) contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence trademarks, contact the corporate legal department at the address shown above or call 1-800-862-4522.

All other trademarks are the property of their respective holders.

Restricted Print Permission: This publication is protected by copyright and any unauthorized use of this publication may violate copyright, trademark, and other laws. Except as specified in this permission statement, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, without prior written permission from Cadence. This statement grants you permission to print one (1) hard copy of this publication subject to the following conditions:

- The publication may be used solely for personal, informational, and noncommercial purposes;

- The publication may not be modified in any way;

- Any copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement; and

- Cadence reserves the right to revoke this authorization at any time, and any such use shall be discontinued immediately upon written notice from Cadence.

Disclaimer: Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. The information contained herein is the proprietary and confidential information of Cadence or its licensors, and is supplied subject to, and may be used only by Cadence customers in accordance with, a written agreement between Cadence and the customer.

Except as may be explicitly set forth in such agreement, Cadence does not make, and expressly disclaims, any representations or warranties as to the completeness, accuracy or usefulness of the information contained in this document. Cadence does not warrant that use of such information will not infringe any third party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from use of such information.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor.

Table of Contents

Verilog Language and Application

Lab 1 Exploring the VeriRISC CPU Design.....5

Lab 2 Modeling an Address Multiplexor7

 Specifications.....7

 Designing the MUX7

 Verifying the MUX design8

Lab 3 Modeling a Data Driver9

 Specifications.....9

 Designing a Data Driver9

 Verifying the Driver Design.....10

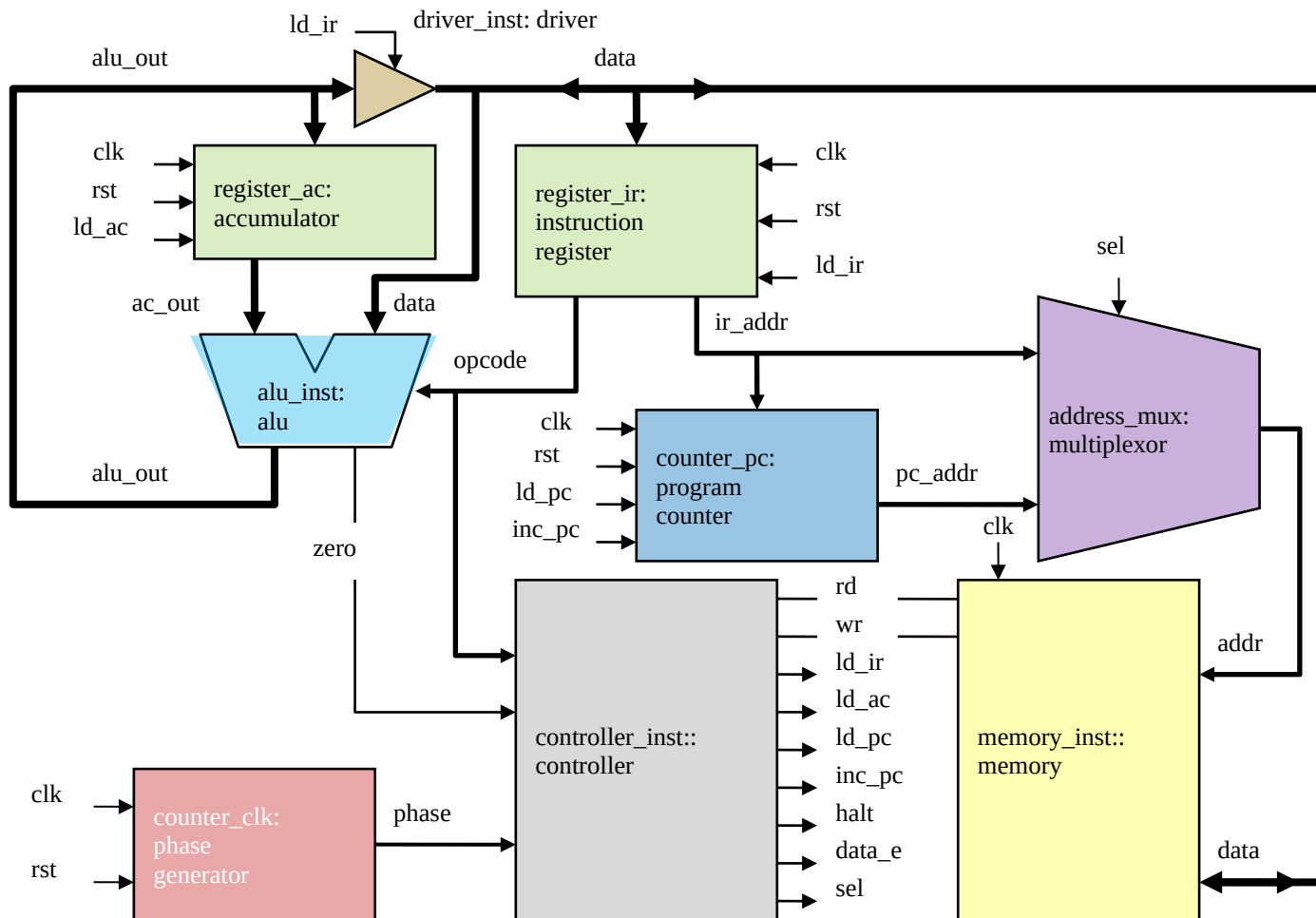
 Verifying the Counter Design10

Lab 1 Exploring the VeriRISC CPU Design

Objective: To understand and become familiar with the VeriRISC lab model.

The VeriRISC model is a very-reduced-instruction-set processor coded in the Verilog HDL. Its instruction consists of a three-bit operation code and a five-bit operand. That restricts its instruction set to eight instructions and its address space to 32 locations. To simplify visualization and debug, it implements the fetch-and-execute cycle in eight phases.

As an exercise, you can easily reduce this to two phases, or with a dual-port memory, to one phase. Input to the model is a clock and a reset, and output from the model is a *halt* signal. Tests determine pass or fail status by the number of clocks the program consumes. **Spend some time and understand the architecture of VeriRISC in this lab. Do not worry about implementation and simulation at this point. You will be implementing and testing this architecture in further labs.**



The upcoming labs are based on the above design. In each of these labs, you develop and verify individual components of the VeryRISC processor like Address Multiplexor, Counter, Controller, Register, ALU, etc. In the lab, you verify the complete design for a few basic instructions under consideration.

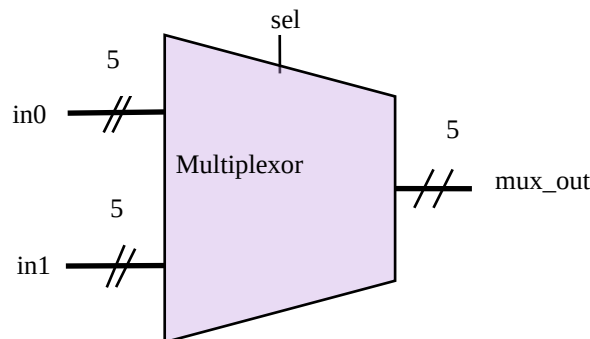


Lab 2 Modeling an Address Multiplexor

Objective: To use basic Verilog constructs to describe a simple Multiplexor design.

In this lab, you create a simple multiplexor design using Verilog constructs and verify it using the supplied testbench.

You will create the MUX design as per the specification and verify it using the provided testbench. Please make sure to use the same variable name as the ones shown in block diagrams.



Specifications

- ♦ The address multiplexor selects between the instruction address during the instruction fetch phase and the operand address during the instruction execution phase.
- ♦ MUX width is parameterized with the default value of 5.
- ♦ If `sel` is `1'b0`, input `in0` is passed to the output `mux_out`.
- ♦ If `sel` is `1'b1`, input `in1` is passed to the output `mux_out`.

Designing the MUX

1. Change to the *lab1-mux* directory and examine the following files.

<code>multiplexor_test.v</code>	Multiplexor test
---------------------------------	------------------

2. Create the *multiplexor.v* file, and using your favorite editor, describe the multiplexor module named “*multiplexor*”.
3. Parameterize the multiplexor input and output widths and assign a default value of 5.

Verifying the MUX design

1. Using the provided test module, check your multiplexor design using the following command with Xcelium™.

```
xrun multiplexor.v multiplexor_test.v (Batch Mode)
```

or

```
xrun multiplexor.v multiplexor_test.v -gui -access +rwc ( GUI Mode)
```

2. You might find it easier to list all the files and simulation options in a text file and pass the file into the simulator using the `-f xrun` option.

```
xrun -f filelist.txt -access rwc
```

You should see the following results.

```
At time 1 sel=0 in0=10101 in1=00000, mux_out=10101
At time 2 sel=0 in0=01010 in1=00000, mux_out=01010
At time 3 sel=1 in0=00000 in1=10101, mux_out=10101
At time 4 sel=1 in0=00000 in1=01010, mux_out=01010
TEST PASSED
```

3. Correct your multiplexor design as needed.

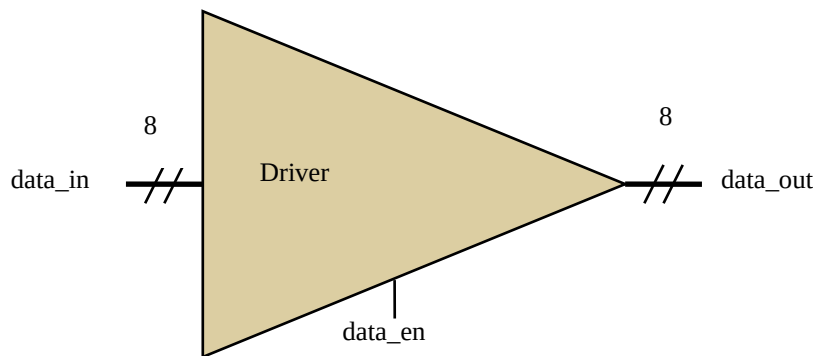


Lab 3 Modeling a Data Driver

Objective: To use a Verilog literal value to describe a parameterized-width bus driver.

The driver output is equal to the input value when enabled (*data_en* is true) and is high-impedance when not enabled (*data_en* is false).

In this lab, you create a data driver as per the specification and verify it using the provided testbench. Please make sure to use the same variable name as the ones shown in block diagrams.



Specifications

- ◆ *data_in* and *data_out* are both parameterized widths of 8-bit.
- ◆ If *data_en* is high, the input *data_in* is passed to the output *data_out*.
- ◆ Otherwise, *data_out* is high impedance.

Designing a Data Driver

1. Change to the *lab2-drvr* directory and examine the following files.

driver_test.v	Driver test
---------------	-------------

2. Create the *driver.v* file, and using your favorite editor, describe the driver module named “*driver*”.
3. Parameterize the driver input and output width and assign a default value of 8.

Verifying the Driver Design

1. Using the provided testbench module, check your driver design using the following command with Xcelium™.

```
xrun driver.v driver_test.v (Batch Mode)
```

or

```
xrun driver.v driver_test.v -gui -access +rwc ( GUI Mode)
```

2. You might find it easier to list all the files and simulation options in a text file and pass the file into the simulator using the `-f xrun` option.

```
xrun -f filelist.txt -access rwc
```

You should see the following results.

```
At time 1 data_en=0 data_in=xxxxxxx data_out=zzzzzzzz
At time 2 data_en=1 data_in=01010101 data_out=01010101
At time 3 data_en=1 data_in=10101010 data_out=10101010
TEST PASSED
```

3. Correct your driver design as needed.
4. And read procedure using *tasks*.
5. In the memory test block, call the appropriate read and write tasks to verify the memory block.

Note: The statement sets that you code read and write procedures given the following task calls.

```
For write: wr=1; rd=0; memory_test.addr=addr; rdata=data; @(negedge
          clk);
```

```
For read:  wr=0; rd=1; memory_test.addr=addr; rdata='bz; @(negedge
          clk) expect(data);
```

Verifying the Counter Design

1. Using the provided test module, check your counter design using the following command with Xcelium™.

```
xrun memory.v memory_test.v (Batch Mode)
```

or

```
xrun memory.v memory_test.v -gui -access +rwc ( GUI Mode)
```

2. You might find it easier to list all the files and simulation options in a text file and pass the file into the simulator using the `-f xrun` option.

```
xrun -f filelist.txt -access rwc
```

You should see the following results.

```
330 addr=11111, exp_data= 00000000, data=00000000
340 addr=11110, exp_data= 00000001, data=00000001
350 addr=11101, exp_data= 00000010, data=00000010
360 addr=11100, exp_data= 00000011, data=00000011
370 addr=11011, exp_data= 00000100, data=00000100
380 addr=11010, exp_data= 00000101, data=00000101
390 addr=11001, exp_data= 00000110, data=00000110
400 addr=11000, exp_data= 00000111, data=00000111
410 addr=10111, exp_data= 00001000, data=00001000
420 addr=10110, exp_data= 00001001, data=00001001
430 addr=10101, exp_data= 00001010, data=00001010
440 addr=10100, exp_data= 00001011, data=00001011
450 addr=10011, exp_data= 00001100, data=00001100
460 addr=10010, exp_data= 00001101, data=00001101
470 addr=10001, exp_data= 00001110, data=00001110
480 addr=10000, exp_data= 00001111, data=00001111
490 addr=01111, exp_data= 00010000, data=00010000
500 addr=01110, exp_data= 00010001, data=00010001
510 addr=01101, exp_data= 00010010, data=00010010
520 addr=01100, exp_data= 00010011, data=00010011
530 addr=01011, exp_data= 00010100, data=00010100
540 addr=01010, exp_data= 00010101, data=00010101
550 addr=01001, exp_data= 00010110, data=00010110
560 addr=01000, exp_data= 00010111, data=00010111
570 addr=00111, exp_data= 00011000, data=00011000
580 addr=00110, exp_data= 00011001, data=00011001
590 addr=00101, exp_data= 00011010, data=00011010
600 addr=00100, exp_data= 00011011, data=00011011
610 addr=00011, exp_data= 00011100, data=00011100
620 addr=00010, exp_data= 00011101, data=00011101
630 addr=00001, exp_data= 00011110, data=00011110
TEST PASSED
```

3. Correct your memory test as needed.

